Polynomial Input Preconditioning for Universal Time Series Forecasting

Anonymous Authors[1]

Abstract

Universal Sequence Preconditioning (USP) is a recent theoretical framework for online sequence prediction that preprocesses observations via a fixed polynomial convolution, yielding hidden-dimension-free sublinear regret for marginally stable linear dynamical systems. This paper investigates whether these techniques transfer empirically to zero-shot patch-based transformer time-series forecasters such as Moirai-2. We propose **polynomial input preconditioning**, where we concatenate the preconditioned signal as an auxiliary input channel while leaving the forecast target unchanged. Architecturally, we only widen the per-patch input projection (12K extra parameters, 0.11% overhead) while leaving the transformer decoder unchanged. On GIFT-Eval (97 configurations, 5 seeds) our method improves geometric-mean MASE by 2.9% over the Moirai-2 Small baseline (cross-seed standard deviation $< 1\%$ of MASE), winning 72 of 97 per-configuration means ($p < 10^{-5}$, paired sign test). Capacity-matched controls confirm that the polynomial content, not the extra parameters, drives the gain. At 100K training steps the benefit grows to 3.9%.

1. Introduction

Universal sequence preconditioning. Marsden & Hazan (2025) proposed Universal Sequence Preconditioning (USP) for online learning of linear dynamical systems. The technique generalises the classical notion of differencing: they suggest preprocessing the observations via a fixed convolution with polynomial coefficients, and prove that this can result in hidden-dimension-free (up to logarithmic factors) sublinear regret with respect to the best LDS predictor. Empirically, USP with fixed Chebyshev or Legendre coefficients reduces prediction error on synthetic linear, nonlinear, and deep-RNN-generated sequences as well as the ETTh1 benchmark.

Does this transfer to zero-shot time-series forecasting? The setting of pretrained foundation models (Woo et al., 2024; Liu et al., 2025; Das et al., 2024; Ansari et al., 2024; Goswami et al., 2024; Auer et al., 2025) differs from USP in two important ways. First, the forecaster is trained offline on a large corpus and deployed for zero-shot prediction over a multi-step horizon, rather than making one-step predictions in an online loop where the true observation is revealed after each step; this distinction is critical because directly applying USP's target-side preconditioning to multi-step rollout leads to catastrophic error accumulation (see below). Second, the training data comes from many heterogeneous sources that may not be well-modelled as finite-dimensional LDSs.

Our approach. If one preconditions the target and trains the model to emit $\hat{y}_t^{\mathrm{pre}}$, recovering the original forecast requires the causal recurrence

$$\hat{y}_t \;=\; \hat{y}_t^{\mathrm{pre}} \;-\; \sum_{k=1}^{d} c_k\, \hat{y}_{t-k},$$

in which each recovered value depends on previously recovered (and therefore noisy) values. Prediction errors compound geometrically across the rollout. Instead, we leave the forecast target unchanged and concatenate the preconditioned signal as an auxiliary input channel alongside the raw target and the observation mask. Because the model never operates in preconditioned space at inference, no inversion is needed and error accumulation is avoided.

A second motivation comes from the architecture itself. Patch-based transformers (Nie et al., 2023; Dosovitskiy et al., 2021) split the input into non-overlapping length-P windows and embed each patch independently, creating a cross-patch information bottleneck: before the first self-attention layer, each patch embedding encodes only P time steps, so multi-patch correlations must be discovered entirely through attention. A polynomial preconditioner that mixes information across the same multi-patch scales be-

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

fore patching can inject cross-patch structure into every embedding at negligible parameter cost.

2. Method

Preconditioning polynomial. Given a normalized input series x and the coefficients $c_0, c_1, \ldots, c_d$ of a degree-d monic polynomial (with $c_0 = 1$), USP convolves the observations: $\sum_{k=0}^{d} c_k\, x_{t-ks}$. We strip the identity term ($c_0 x_t$, which the model already sees) and feed only the residual as an auxiliary channel:

$$r_t \;=\; \sum_{k=1}^{d} c_k\, x_{t-ks}, \tag{1}$$

where s is a stride parameter set equal to the patch size P. Our default is the monic Chebyshev polynomial of the first kind, $\tilde{T}_d$, with $d = 4$. For $P = 16$ the coefficients are $(c_1, \ldots, c_4) = (0, -1, 0, \frac{1}{8})$, so $r_t = -x_{t-32} + \frac{1}{8}\, x_{t-64}$.

Preconditioning channel. The residual r is partitioned into patches $\mathbf{r}_i$ aligned with the data patches $\mathbf{p}_i$. In Moirai-2, each patch passes through a per-patch input projection (a two-layer residual MLP with SiLU activation) that maps the concatenation of the data patch and an observation mask to an embedding vector. We widen this projection to accept a third input, the preconditioning patch: the input dimension grows from $2P$ to $3P$, adding 12K parameters (0.11% of Moirai-2 Small). The transformer decoder is unchanged. We refer to this default ($d = 4$ Chebyshev, no dropout) as `d4`, and to the same architecture trained with 10% per-patch channel dropout as `d4_dropout`.

Training and inference. We train from scratch using the Moirai-2 Small architecture (Liu et al., 2025) on a publicly available subset of the LOTSA corpus (Woo et al., 2024) (the full official training data is not publicly released). Due to the smaller data budget, we train for 10K steps with 1K-step warmup (the official model uses 100K steps with 10K warmup). All other settings (optimizer, loss, precision) follow the official protocol. The only architectural modification is the wider input projection; for `d4_dropout`, the preconditioning channel is independently zeroed with probability 0.1 per patch during training and always provided at inference. At inference, the model predicts multiple future patches per step; for longer horizons, predicted patches are appended to the context and the decoder is re-run with the preconditioning residual recomputed over the extended context.

3. Experiments

Setup. We train Moirai-2 Small (Liu et al., 2025) (11.4M params, 6-layer transformer, P=16) from scratch on the LOTSA corpus (Woo et al., 2024) for 10K steps (batch 256, AdamW, cosine LR 10^{-3}, 1K warmup, bf16). Our Baseline is a best-effort from-scratch retrain of Moirai-2 Small under this protocol; the official Moirai 2.0-R-Small release used Salesforce's full, undisclosed training compute, so our retrain does not (and cannot) reproduce its absolute leaderboard numbers. All conditions share identical data, optimiser, seeds, and compute, so the effects we report are controlled relative improvements. Evaluation uses GIFT-Eval (Aksu et al., 2024) (97 dataset×horizon configs) at context length 4,000 and the independent FEV-Bench (Shchur et al., 2025) (100 tasks). We report normalized MASE following the GIFT-Eval leaderboard convention: each configuration's MASE is divided by the seasonal naive baseline's MASE, then aggregated via geometric mean. We run 5 seeds (0, 1, 2, 7, 42) and report cross-seed mean and standard deviation. See Appendix A for full details.

Algorithm 1 Training with Polynomial Input Preconditioning

Require: Patched sequence $\mathbf{p}_{1:N}$ with masks $\mathbf{m}_{1:N}$; Chebyshev coefficients $c_{1:d}$ (fixed); stride $s{=}P$; channel dropout probability p_{drop}.

1: z**-score:** normalise patches using context statistics.
2: **for** $i = 1$ **to** N **do**
3: $\quad \mathbf{r}_i \leftarrow \sum_{k=1}^{d} c_k \cdot \tilde{\mathbf{p}}_{i-k}$ {Preconditioning residual}
4: $\quad$ With probability p_{drop}, set $\mathbf{r}_i \leftarrow \mathbf{0}$. {Channel dropout}
5: **end for**
6: **Embed:** $\mathbf{e}_i \leftarrow \text{InputProj}([\tilde{\mathbf{p}}_i;\, \mathbf{m}_i;\, \mathbf{r}_i])$ for all i.
7: **Decode:** $\mathbf{h}_{1:N} \leftarrow \text{CausalTransformer}(\mathbf{e}_{1:N})$.
8: **Predict:** $\hat{\mathbf{y}}_i \leftarrow \text{OutputProj}(\mathbf{h}_i)$ for prediction-window patches.
9: $\mathcal{L} \leftarrow \text{QuantileLoss}(\hat{\mathbf{y}},\, \tilde{\mathbf{p}})$ over prediction window.
10: Update all trainable parameters via $\nabla\mathcal{L}$.

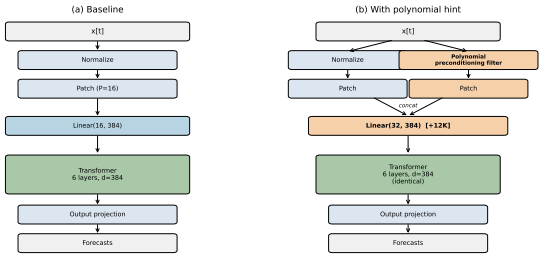

Figure 1. Moirai-2.0 architecture: (a) standard patch transformer; (b) with polynomial input preconditioning. Only the per-patch input projection is wider; the transformer decoder is unchanged. Total overhead: 12K parameters (0.11%).

Conditions. We compare four conditions, all sharing the same training data, optimizer, and seeds. **Baseline** is unmodified Moirai-2 (no preconditioning channel). **`d4`** and

Algorithm 2 Inference with Polynomial Input Preconditioning

Require: History $x_{1:T}$; prediction length H; Chebyshev coefficients $c_{1:d}$; stride $s{=}P$.
Ensure: Point forecasts $\hat{x}_{T+1:T+H}$.
1: **z-score:** compute μ, σ from $x_{1:T}$. Set $\tilde{x}_t \leftarrow (x_t - \mu)/\sigma$.
2: **Patchify:** form context patches $\tilde{\mathbf{p}}_i$ with $\mathbf{m}_i = \mathbf{1}$; zero-fill prediction patches with $\mathbf{m} = \mathbf{0}$.
3: **Residual:** $\mathbf{r}_i \leftarrow \sum_{k=1}^{d} c_k \cdot \tilde{\mathbf{p}}_{i-k}$ for all patches. {No dropout}
4: **Embed, decode, predict:** run InputProj $\rightarrow$ CausalTransformer $\rightarrow$ OutputProj.
5: Read median forecasts from the last context patch for the next several patches.
6: **if** horizon covered **then**
7: **Return** $\hat{x}_{T+1:T+H} \leftarrow \sigma \cdot \hat{\mathbf{y}}^{(0.5)}_{1:H} + \mu$.
8: **else**
9: Fill predicted patches into context, recompute $\mathbf{r}$, and repeat.
10: **end if**

Table 1. Main results on GIFT-Eval (5 seeds). MASE is averaged per configuration across seeds, then aggregated via geometric mean. Wins: configurations where the method's mean MASE < Baseline's (out of 97); p: two-sided paired sign test.

Method	MASE	$\Delta\%$	Wins	p
d4_dropout (Ours)	**0.837**	**−2.9**	**72/97**	$< 10^{-5}$
d4 (Ours)	0.838	−2.8	72/97	$< 10^{-5}$
Zero (capacity ctrl)	0.858	−0.5	54/97	0.31
Baseline	0.862	—	—	—
Duplicate (capacity ctrl)	0.865	+0.4	39/97	0.07

d4_dropout add the polynomial preconditioning channel of §2 (without and with 10% per-patch channel dropout). **Zero** and **Duplicate** are capacity-matched controls: they share d4_dropout's widened input projection and 12K extra parameters, but feed the third channel either all zeros (Zero) or a copy of the data channel (Duplicate). They isolate whether any gain comes from the polynomial content or merely from the extra capacity.

3.1. Main Results

Both d4 and d4_dropout improve over Baseline on every seed (Appendix B); their cross-seed means are within 0.1% of each other, so the bulk of the gain comes from the polynomial content rather than the dropout. Zero and Duplicate share d4_dropout's architecture and 12K-parameter overhead, isolating the causal role of polynomial content: Duplicate hurts (39/97 wins, below chance) and Zero is not significant (54/97, $p = 0.31$). The improvement is driven by the polynomial content, not the extra parameters.

Table 2. FEV-Bench: d4_dropout (5-seed mean) vs single-seed Baseline. SQL is FEV-Bench's scaled quantile loss. Cross-seed std for ours is ≤ 0.007 on both metrics.

Method	MASE	SQL	Wins
d4_dropout (Ours)	**1.2532**	**1.0236**	**72/100**
Baseline	1.2908	1.0562	—
$\Delta\%$ vs BL	−2.91%	−3.09%	

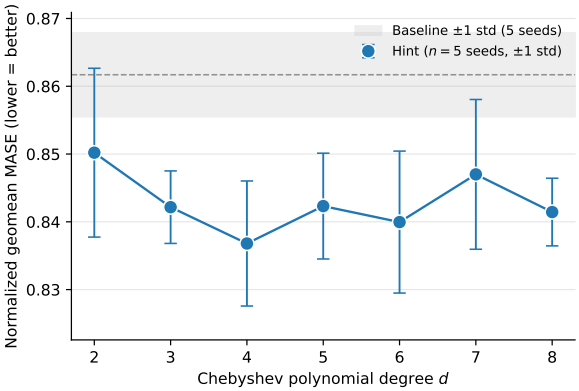

Figure 2. Chebyshev polynomial degree sweep (10K steps, stride $s{=}16$, 10% channel dropout). Filled blue circles are 5-seed means with ±1 std error bars (seeds 0, 1, 2, 7, 42). Grey band: Baseline 5-seed mean ±1 std. Every $d \geq 2$ beats the baseline; the curve is shallow with $d \in \{3, 4, 5, 6, 8\}$ within ∼0.6% of each other. Full ablations over stride, basis, and channel dropout are in Appendix E.

FEV-Bench. On the independent FEV-Bench (100 tasks; full degree sweep in Appendix N), d4_dropout improves over Baseline by similar margins as on GIFT-Eval, confirming the benefit is not benchmark-specific.

3.2. Ablations

Figure 2 sweeps Chebyshev polynomial degree $d \in \{2, \ldots, 8\}$ over 5 seeds. Every $d \geq 2$ beats the baseline; the curve is shallow—$d \in \{3, 4, 5, 6, 8\}$ are all within ∼0.6% of each other—so $d = 4$ is representative rather than uniquely optimal. The extremes $d = 2$ and $d = 7$ trail the rest. Appendix E reports stride/basis/dropout ablations ($s = P = 16$ optimal; Chebyshev outperforms EMA/Legendre/finite-differences; dropout inverted-U with 10% optimum) and inference-time content ablations confirming the model exploits the specific polynomial structure rather than generic extra capacity.

3.3. Extended Training and Robustness

At 100K optimizer steps (Table 3, 5 seeds) the improvement grows to 3.9% while both capacity controls now actively hurt (Duplicate +0.9%, Zero +1.7%); d4_dropout also shows much lower cross-seed variance ($\sigma = 0.009$ vs 0.019 for BL). The training-loss curves (Fig. 3) show a consistent separation that widens in the later stages of training (Appendix C).

Table 3. 100K training budget, 5 seeds. Normalized geomean MASE.

Method	MASE	Δ% vs BL	Std
d4_dropout (Ours)	**0.8443**	−3.88%	0.009
Baseline	0.8784	—	0.019
Duplicate (cap. ctrl)	0.8860	+0.86%	0.008
Zero (cap. ctrl)	0.8937	+1.73%	0.036

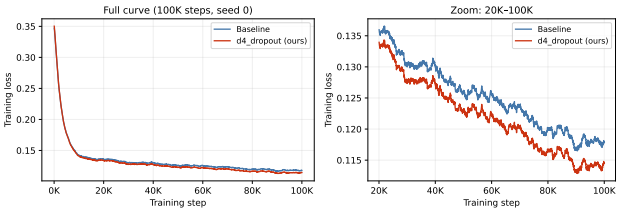

Figure 3. Training loss on LOTSA (seed 0, 100K steps, EMA-smoothed). Left: full curve. Right: zoom on steps 20K–100K. d4_dropout consistently tracks below Baseline throughout training, with the gap widening in the later stages.

Domain, frequency, and context. The benefit concentrates in high-resolution domains whose natural periodicities match the polynomial's 32/64-step lookback: Demographics (+9.8%), Transport (+6.9%), Energy (+4.0%); Weather (+0.3%) and Healthcare (+0.2%) are essentially unchanged (Appendix F). The benefit is approximately constant ($\sim 5\%$) across context lengths 512–4,000; d4_dropout at context 2,000 outperforms the baseline at context 4,000 (Appendix H).

Horizon stratification. On short horizons (PL ≤ 60) d4 and d4_dropout are within 0.4% of each other (Table 4; full analysis in Appendix G); on medium and long horizons (PL ≥ 480) d4_dropout beats d4 by 0.7–0.85% relative MASE—the direction the dropout-as-mismatch-regulariser story predicts but small relative to the $\sim 4\%$ HD-vs-Baseline gap.

4. Related Work

Time-series foundation models. Transformer-based universal forecasters have proliferated (Woo et al., 2024; Liu et al., 2025; Das et al., 2024; Ansari et al., 2024; Goswami et al., 2024; Auer et al., 2025), trained on large, heterogeneous corpora for zero-shot forecasting across unseen domains. Recent work questions whether current TSFMs meaningfully exceed simple classical baselines in the zero-shot regime (Zhang et al., 2025)—a critique that motivates exactly our regime of interest: strengthening compact foundation models at low parameter cost. Most architectural innovation in this space (rotary position embeddings, attention variants, mixture-of-experts (Liu et al., 2025)) targets the transformer stack; input preconditioning is orthogonal, acting on the input representation before any attention layer.

Table 4. Horizon stratification of d4 vs d4_dropout vs Baseline. 5-seed cross-config geomean MASE; Δ% relative to BL within each bin.

Term	n	BL	d4 (Δ)	d4_dropout (Δ)
short	55	0.840	0.825 (−1.8%)	0.828 (−1.4%)
medium	21	0.859	0.830 (−3.4%)	**0.823** (−4.2%)
long	21	0.924	0.880 (−4.7%)	**0.874** (−5.3%)

Patching and cross-patch information. Patch-based architectures (Nie et al., 2023; Dosovitskiy et al., 2021) dominate modern TSFMs. PatchTST uses overlapping patches to partially mitigate the cross-patch bottleneck; our approach achieves the same effect without increasing FLOPs or decoder parameters, and is compatible with both overlapping and non-overlapping patch schemes.

Spectral methods in sequence modelling. Hazan et al. (2017; 2018) developed spectral-filtering algorithms for online LDS prediction with hidden-dimension-free guarantees; Marsden & Hazan (2025) unify this line via polynomial preconditioning. Classical polynomial approximation with Chebyshev polynomials has a long tradition (Parks & McClellan, 1972; Oppenheim et al., 1999); our contribution is to bring it to bear on the input representation of a pretrained foundation model rather than as a post-hoc smoother.

5. Discussion and Conclusion

Polynomial input preconditioning adapts the ideas behind Marsden & Hazan (2025) from online LDS prediction to zero-shot patch-based time-series transformers. Concatenating the preconditioned signal as an auxiliary channel avoids the error accumulation of target-side preconditioning at multi-step horizons, and adds only 0.11% parameters. One limitation is that at Base (46M) scale the per-seed gain is inconsistent, suggesting that sufficient attention capacity can absorb the inductive bias that preconditioning provides at smaller scale.

Impact statement. This paper advances the field of Machine Learning; we identify no societal consequences requiring specific highlighting.

References

Aksu, T., Woo, G., Liu, J., Liu, X., Liu, C., Savarese, S., Xiong, C., and Sahoo, D. GIFT-Eval: A benchmark for general time series forecasting model evaluation. *arXiv preprint arXiv:2410.10393*, 2024.

Ansari, A. F., Stella, L., Turkmen, C., Zhang, X., Mercado, P., Shen, H., Shchur, O., Rangapuram, S. S., Arango, S. P., Kapoor, S., et al. Chronos: Learning the language of time series. In *ICML*, 2024.

Auer, A., Podest, P., Klotz, D., Böck, S., Klambauer, G., and Hochreiter, S. TiRex: Zero-shot forecasting across long and short horizons. In *ICML Workshop on Foundation Models for Structured Data (FMSD)*, 2025. URL https://openreview.net/forum?id=cWrbpOZGRa.

Das, A., Kong, W., Leber, A., Mathews, R., and Sen, R. A decoder-only foundation model for time-series forecasting. In *ICML*, 2024.

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., and Houlsby, N. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*, 2021.

Goswami, M., Szafer, K., Choudhry, A., Cai, Y., Li, S., and Dubrawski, A. Moment: A family of open time-series foundation models. *arXiv preprint arXiv:2402.03885*, 2024.

Hazan, E., Singh, K., and Zhang, C. Learning linear dynamical systems via spectral filtering. In *NeurIPS*, 2017.

Hazan, E., Lee, H., Singh, K., Zhang, C., and Zhang, Y. Spectral filtering for general linear dynamical systems. In *NeurIPS*, 2018.

Liu, C., Aksu, T., Liu, J., Liu, X., Yan, H., Pham, Q., Savarese, S., Sahoo, D., Xiong, C., and Li, J. Moirai 2.0: When less is more for time series forecasting. *arXiv preprint arXiv:2511.11698*, 2025.

Marsden, A. and Hazan, E. Universal sequence preconditioning. *arXiv preprint arXiv:2502.06545*, 2025.

Nie, Y., Nguyen, N. H., Sinthong, P., and Kalagnanam, J. A time series is worth 64 words: Long-term forecasting with transformers. In *ICLR*, 2023.

Oppenheim, A. V., Schafer, R. W., and Buck, J. R. *Discrete-Time Signal Processing*. Prentice Hall, 1999.

Parks, T. W. and McClellan, J. H. Chebyshev approximation for nonrecursive digital filters with linear phase. *IEEE Transactions on Circuit Theory*, 1972.

Shchur, O., Turkmen, C., Ansari, A. F., Stella, L., and Rangapuram, S. S. FEV-Bench: A unified benchmark for few-shot time series forecasting evaluation. *arXiv preprint arXiv:2509.26468*, 2025.

Woo, G., Liu, C., Kumar, A., Xiong, C., Savarese, S., and Sahoo, D. Unified training of universal time series forecasting transformers. In *ICML*, 2024.

Zhang, Y., Zeng, Q., Zhang, Y., Xu, Z., Zheng, M., Gao, C., Jiang, M., and Zheng, Z. Are time series foundation models ready for zero-shot forecasting? In *ICML Workshop on Foundation Models for Structured Data (FMSD)*, 2025. URL https://openreview.net/forum?id=kQDzqIkXLO.

A. Detailed Methodology

A.1. Training Protocol

All experiments use identical training infrastructure on NVIDIA H100/A100 GPUs. The complete configuration:

Parameter	Value
Architecture	Moirai-2 Small (6 layers, $d = 384$, $d_{\text{ff}} = 1024$, $P = 16$)
Total parameters	11.4M (baseline), 11.4M + 12K (`d4_dropout`)
Training data	LOTSA v1, Moirai-2 config (weighted, variate-proportional)
Steps (10K runs)	100 epochs $\times$ 100 batches = 10,000 steps
Steps (100K runs)	1,000 epochs $\times$ 100 batches = 100,000 steps
Batch size	256
Optimizer	AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.98$, $\epsilon = 10^{-6}$, wd $= 0.1$)
LR schedule	Cosine annealing, peak $\eta = 10^{-3}$, min $\eta = 0$
Warmup	1,000 steps (linear)
Precision	bf16 mixed precision
Anomaly filtering	z-score threshold = 8.0, variance ratio = 0.0 (disabled)
Random seeds	0, 1, 2, 7, 42 (5 seeds for main results)

A.2. Evaluation Protocol

All evaluations use the GIFT-Eval benchmark (Aksu et al., 2024) with 97 dataset$\times$horizon configurations:

- Context length: 4,000 tokens (maximum supported)
- Patch size: 32 for most frequencies, 8 for quarterly (following Moirai conventions)
- Batch size: 64
- Metric: Normalized MASE (each configuration's MASE divided by the seasonal naive baseline's MASE)
- Aggregation: geometric mean across 97 configurations
- Reported quantities: cross-seed mean and standard deviation, per-seed MASE, and per-configuration win counts (out of 97 per seed; $5 \times 97 = 485$ when pooled across seeds)

A.3. Preconditioning Channel Implementation

The monic Chebyshev polynomial $\tilde{T}_d(z)$ is expanded in the monomial basis to obtain the coefficients $c_0, \ldots, c_d$ with $c_0 = 1$. The preconditioning residual for time step t is:

$$r_t = \left(\sum_{k=0}^{d} c_k \, x_{t-ks} \right) - x_t$$

This is computed over the full (normalized, zero-mean unit-variance) time series before patching. Unobserved time steps are masked to zero. During training with channel dropout p, each patch's preconditioning channel independently has probability p of being replaced with zeros. At inference, the full preconditioning channel is always provided.

The preconditioning channel is concatenated with the target and mask channels before the input projection:

$$\mathbf{e}_i = \text{ResBlock}_{(2+n_h)P \to d}\big([\mathbf{p}_i; \mathbf{m}_i; \mathbf{r}_i^{(1)}; \ldots; \mathbf{r}_i^{(n_h)}]\big)$$

where $n_h = 1$ for single-scale (`d4_dropout`) and $n_h = 2$ for multi-scale (MSHD10). The ResBlock consists of:

$$\mathbf{z} = \text{SiLU}(W_1 \mathbf{x} + b_1) \qquad W_1 \in \mathbb{R}^{d \times (2+n_h)P} \tag{2}$$

$$\mathbf{o} = W_2 \mathbf{z} + b_2 + W_3 \mathbf{x} + b_3 \qquad W_2 \in \mathbb{R}^{d \times d},\ W_3 \in \mathbb{R}^{d \times (2+n_h)P} \tag{3}$$

Only W_1 and W_3 are wider than the baseline. W_2 and all downstream layers are identical.

TransformerDecoder. Moirai-2 Small uses a 6-layer decoder-only transformer with $d_{\text{model}} = 384$, $d_{\text{ff}} = 1024$, 6 attention heads, causal multi-head self-attention with rotary position embeddings (RoPE), SwiGLU feed-forward blocks, and RMSNorm. The output projection is another ResBlock mapping $\mathbb{R}^d \to \mathbb{R}^{K \times |\mathcal{Q}| \times P}$ where K=4 is the multi-token horizon and $|\mathcal{Q}|$=9 the number of quantile levels.

B. Per-Seed Results

Table 5. `d4_dropout` vs. Baseline: per-seed pairwise comparison (10K steps, $\eta = 10^{-3}$).

Seed	BL MASE	**d4_dropout** MASE	Δ%	Wins/97
0	0.8666	0.8234	+4.99%	66
1	0.8503	0.8465	+0.44%	54
2	0.8630	0.8391	+2.77%	70
7	0.8676	0.8320	+4.11%	78
42	0.8614	0.8430	+2.13%	61
Mean	**0.8617**	**0.8368**	+2.89%	**329/485**
Std	0.0062	0.0083	1.58%	—

Per-seed MASE for all four conditions:

Table 6. Per-seed results. `d4_dropout` achieves lower MASE than all controls on every seed.

Seed	Baseline	**d4_dropout (Ours)**	Zero	Duplicate
0	0.8666	**0.8234**	0.8586	0.8777
1	0.8503	**0.8465**	0.8738	0.8546
2	0.8630	**0.8391**	0.8425	0.8662
7	0.8676	**0.8320**	0.8573	0.8602
42	0.8614	**0.8430**	0.8569	0.8661
Mean	0.8617	**0.8368**	0.8578	0.8650
Std	0.0062	0.0083	0.0102	0.0081

When averaging each configuration's MASE across all 5 seeds, `d4_dropout` wins 72/97 configurations. `d4_dropout` wins 3+ out of 5 seeds on 76.3% of configurations, and wins all 5 seeds on 23 configurations. Only 2 configurations see the baseline win all 5 seeds.

C. Training Loss Analysis

Table 7. Final training loss across 5 seeds at 10K steps ($\eta = 10^{-3}$). Training loss differences are small ($\sim$1%), while eval MASE differences are much larger ($\sim$3%), confirming preconditioning acts as a regularizer.

Method	s0	s1	s2	s7	s42	Mean	Std	Eval MASE
Baseline	.1366	.1365	.1358	.1366	.1365	.1364	.0003	0.8617
d4_dropout (ours)	.1354	.1350	.1350	.1353	.1343	**.1350**	.0004	**0.8368**
Zero	.1378	.1369	.1363	.1366	.1353	.1366	.0008	0.8578
Duplicate	.1376	.1366	.1370	.1365	.1355	.1366	.0007	0.8650
d4_dropout Δ% vs BL						-1.0%		-2.9%

D. Capacity Controls

The Zero and Duplicate conditions serve as capacity controls: they have the same architecture and parameter count as `d4_dropout`, differing only in the content of the extra channel.

D.1. 5-Seed Results at 10K Steps

- **Duplicate channel hurts**: win rate 44.7%, worse than the 50% expected by chance.

Table 8. Final training loss across seeds at 100K steps ($\eta = 10^{-3}$). `d4_dropout` maintains lower training loss on every seed.

Method	s0	s1	s2	s7	s42	Mean	Std	Eval MASE
Baseline	.1185	.1185	.1190	.1194	.1179	.1187	.0005	0.8784
`d4_dropout` (ours)	.1144	.1170	.1155	.1172	.1155	**.1159**	.0010	**0.8443**
Zero	.1186	.1188	.1184	.1188	.1187	.1187	.0002	0.8937
Duplicate	.1192	.1193	.1185	.1197	.1182	.1190	.0006	0.8860
MSHD10	.1147	—	.1145	.1145	—	.1146	.0001	0.8378[†]
`d4_dropout` $\Delta\%$ vs BL						-2.4%		-3.9%

[†]MSHD10 eval mean over 3 available seeds (0, 2, 7).

Table 9. Train loss vs eval MASE improvement (5-seed mean, `d4_dropout` over baseline). The eval improvement consistently exceeds the training loss improvement, indicating a regularization effect.

	Train loss $\Delta\%$	Eval MASE $\Delta\%$	Amplification
10K steps (5 seeds)	-1.0%	-2.9%	$2.9\times$
100K steps (5 seeds)	-2.4%	-3.9%	$1.6\times$

- **Zero channel is neutral**: win rate 54.0%, tiny effect (-0.45%).
- **`d4_dropout` vs. Duplicate**: 336/485 wins (`d4_dropout` beats Duplicate on every seed).
- **`d4_dropout` vs. Zero**: 323/485 wins (`d4_dropout` beats Zero on every seed).

D.2. 5-Seed Results at 100K Steps

At 100K steps, the capacity controls that were near-neutral at 10K now actively hurt: Duplicate degrades by 0.9% and Zero by 1.7% relative to the baseline. See Appendix K for the full table.

E. Ablations

All ablations use seed 0 with 10% channel dropout unless otherwise noted; the polynomial-degree sweep is the exception and is reported over 5 seeds each.

E.1. Polynomial Degree (Chebyshev, $s = 16$, 10% dropout)

Every $d \geq 2$ beats the baseline. The mean curve is shallow: $d = 4$ achieves the lowest geomean MASE but $d \in \{3, 5, 6, 8\}$ are within roughly one seed-to-seed standard deviation of it. The extreme $d = 2$ has only one non-trivial residual lag (at $2P$). For odd d, the parity of the monic Chebyshev polynomial makes every other residual coefficient vanish, so $d \in \{3, 5, 7\}$ use the same residual-lag supports as $d - 1 \in \{2, 4, 6\}$ but with different weights. With single-seed evaluation the apparent optimum was sharper and even-versus-odd ordering looked systematic; the 5-seed estimates show that ordering is not robust to seed variation, while the qualitative "every degree helps" conclusion is.

E.2. Stride (Chebyshev $d = 4$, 10% dropout)

Stride $s = 16$ (equal to patch size P) is optimal: the nonzero coefficients align exactly with patch boundaries, providing maximally informative cross-patch lookback.

E.3. Basis ($d = 4$, $s = 16$, 10% dropout)

The empirical ordering directly matches the L^∞-norm prediction: the basis that minimises $\sup_{[-1,1]} |p|$ wins. The explicit residuals for $d = 4$ are:

- **Monic Chebyshev** ($\tilde{T}_4(z) = z^4 - z^2 + 1/8$): $r_t = -x_{t-2P} + \frac{1}{8} x_{t-4P}$. Sup-norm $= 1/8 = 0.125$, the minimax-optimal value.

Table 10. Capacity control experiments (5 seeds, $\eta = 10^{-3}$, 10K steps). Same architecture width, different channel content.

Method	Content	Mean MASE	Std	Wins/485
d4_dropout	Monic Chebyshev $\tilde{T}_4(B^{16})$ residual	**0.8368**	0.0083	**329**
Zero	All zeros	0.8578	0.0102	262
Baseline	No extra channel	0.8617	0.0062	—
Duplicate	Copy of input	0.8650	0.0081	217

Degree	Residual lags	MASE (mean $\pm$ std, n=5)	Δ% vs BL
2	1 (at $2P$)	0.8502 ± 0.0125	-1.33%
3	1 (at $2P$)	0.8422 ± 0.0053	-2.27%
4	2 (at $2P$, $4P$)	$\mathbf{0.8368 \pm 0.0092}$	$\mathbf{-2.89\%}$
5	2 (at $2P$, $4P$)	0.8423 ± 0.0078	-2.25%
6	3 (at $2P$, $4P$, $6P$)	0.8400 ± 0.0105	-2.52%
7	3 (at $2P$, $4P$, $6P$)	0.8470 ± 0.0110	-1.71%
8	4 (at $2P$, $4P$, $6P$, $8P$)	0.8414 ± 0.0050	-2.36%
Baseline	—	0.8617 ± 0.0062	—

- **Monic Legendre** ($\tilde{P}_4(z) = z^4 - \frac{6}{7}z^2 + \frac{3}{35}$): $r_t = -\frac{6}{7}x_{t-2P} + \frac{3}{35}x_{t-4P}$. Same sparsity pattern but sup-norm $= 8/35 \approx 0.229$, a $1.83\times$ gap vs Chebyshev.

- **EMA** (low-pass): geometrically decaying weights $a_k \propto \alpha^k$, truncated to $d = 4$ terms. Not polynomial-based; no minimax guarantee.

- **Finite difference** (high-pass): $p(z) = (z-1)^4$, giving $r_t = -4x_{t-P} + 6x_{t-2P} - 4x_{t-3P} + x_{t-4P}$. Sup-norm $= 2^4 = 16$, amplifying rather than compressing spectral content.

The Chebyshev–Legendre gap of ~ 1.7 percentage points is consistent with the $1.83\times$ theoretical sup-norm ratio. All polynomial bases except finite differences beat Baseline, confirming that cross-patch information injection is the load-bearing mechanism; the specific coefficients determine how much benefit is extracted.

E.4. Channel Dropout Rate (Chebyshev $d = 4$, $s = 16$)

E.5. Learned Coefficients (seed 0)

The two learned solutions converge to distinct basins. Both beat the baseline but are slightly worse than fixed Chebyshev (51/97 head-to-head wins for fixed).

E.6. Latent Preconditioning (seed 0)

Applying the polynomial after the input projection (in latent space) rather than before patching:

Latent preconditioning is catastrophic. The polynomial must operate on the raw time series, not on learned embeddings.

F. Domain and Frequency Analysis

The benefit is consistent with the mechanism: at stride $s = 16$ and patch size $P = 16$, the polynomial looks back 32 and 64 time steps. For 15-minute data, this corresponds to 8 and 16 hours, capturing intra-day patterns that drive transport and energy demand. For daily weather data, the lookback spans 32 and 64 days, too long to capture the relevant weekly/seasonal patterns.

Prediction horizon. The benefit increases with prediction length: $+1.4\%$ for short horizons to $+5.1\%$ for long horizons. Longer horizons require more temporal structure, which the preconditioning channel provides.

Stride	Lookback	MASE	Δ% vs BL	Wins/97
16 (= P)	32, 64 steps	**0.8234**	**−4.99%**	**66**
4	8, 16 steps	0.8406	−3.00%	57
8	16, 32 steps	0.8518	−1.71%	62
32	64, 128 steps	0.8484	−2.10%	63
8+16 (dual)	16, 32, 64 steps	0.8389	−3.20%	62

Basis	MASE	Δ%	Wins vs BL	Wins vs Cheby
Chebyshev	**0.8234**	**−4.99%**	66/97	—
EMA	0.8329	−3.89%	65/97	41/97
Legendre $d = 4$	0.8400	−3.07%	62/97	30/97
Legendre $d = 6$	0.8397	−3.10%	64/97	50/97
Finite Diff	0.8569	−1.12%	54/97	29/97
Duplicate	0.8583	−0.96%	42/97	27/97

G. Horizon Analysis: `d4` vs `d4_dropout`

The preconditioning polynomial for `d4`/`d4_dropout` (d=4, s=16) reduces to $r_t = -x_{t-32} + (1/8)\, x_{t-64}$. At inference, GIFT-Eval pads the prediction window with zeros before the first decode pass, so for a predicted position t each lag x_{t-ks} is real if $t - ks$ falls in the context window and zero otherwise; for `prediction_length` > `num_predict_token` the model is unrolled in chunks and the lags in the second and later chunks are populated by the model's own predictions. The threshold below which the preconditioned signal is fully real is $d \cdot s = 64$ steps. We test whether `d4_dropout`'s channel-dropout regularisation buys robustness on long horizons that `d4` lacks.

We compute, for each of 97 GIFT-Eval configs and each of 5 seeds, normalised MASE for Baseline, `d4` (Chebyshev $d = 4$, $s = 16$, no dropout), and `d4_dropout` (same with 10% patch-level channel dropout). Each bin's reported MASE is the cross-seed mean of the per-seed cross-config geometric mean. Δ% is relative to the same-bin Baseline. "Wins" counts configs (out of bin n) where the method's 5-seed mean beats the Baseline's; "`d4` better than `d4_dropout`" counts configs where `d4`'s 5-seed mean beats `d4_dropout`'s.

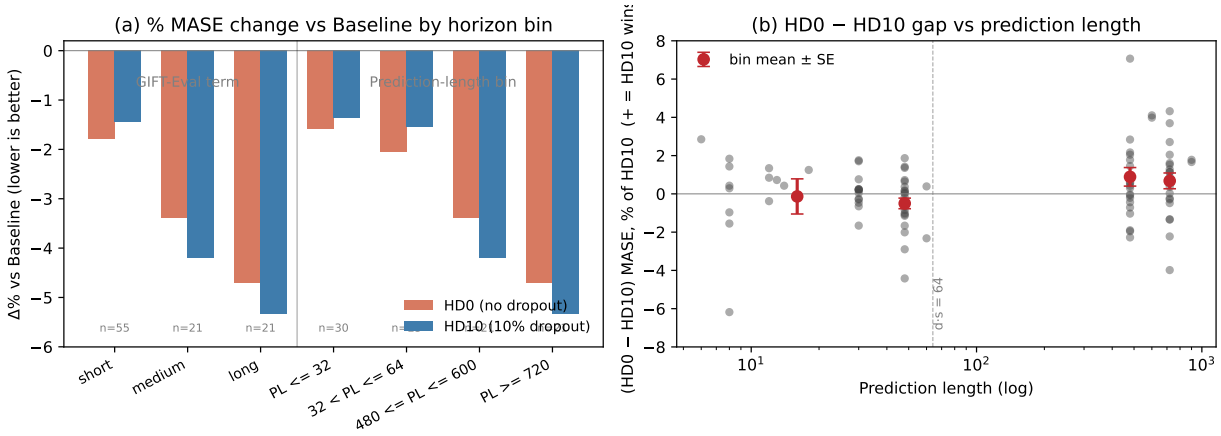

Figure 4. (a) Per-bin Δ% vs Baseline for `d4` (no dropout) and `d4_dropout` (10% dropout). `d4` is comparable on short horizons but trails `d4_dropout` on medium/long. (b) Per-config `d4` − `d4_dropout` MASE gap (as a percentage of `d4_dropout`) plotted against prediction length on a log axis; positive values mean `d4_dropout` wins. The dashed line marks the $d \cdot s = 64$ threshold below which the preconditioned signal is fully computable from context. Bin means with ±1 standard error are red.

What the data say. The direction of the `d4`–`d4_dropout` gap is consistent with the train/inference mismatch story: across the $d \cdot s = 64$ threshold the gap flips from slightly favouring `d4` (−0.36% overall on short, −0.50% on $32 < \text{PL} \leq 64$) to favouring `d4_dropout` (+0.85% on medium, +0.66% on long). However, the magnitude is small ($\leq 0.85\%$ relative MASE) compared to the $\sim$ 4–5% HD-vs-Baseline gap on the same long horizons. Most of the polynomial-preconditioning benefit transfers from training to inference even without dropout. The practical implication is that `d4_dropout` is the safer

Dropout	MASE	Δ% vs BL	Wins/97
0% (d4)†	0.8378	-3.5%	—
2%	0.8388	-3.20%	63
3%	0.8497	-1.95%	55
10%	**0.8234**	$\mathbf{-4.99\%}$	**66**
20%	0.8422	-2.81%	67
30%	0.8559	-1.23%	61

†d4 MASE is the 5-seed mean. d4 and d4_dropout are essentially tied; dropout helps marginally.

Initialization	Learned coefficients	MASE	Δ% vs BL
Chebyshev init	$[-0.026, -1.032, -0.123, 0.007]$	0.8332	-3.85%
Zero init	$[0.308, 0.177, 0.057, 0.045]$	0.8358	-3.57%
Fixed Chebyshev	$[0, -1, 0, 0.125]$ (not learned)	0.8234	-4.99%

default—especially since long horizons are precisely where the polynomial-preconditioning effect is largest in absolute terms—but the size of the dropout-mismatch effect alone does not motivate adopting d4_dropout over d4 in isolation.

H. Context Length

Notably, d4_dropout at context length 2,000 (MASE 0.8331) outperforms the baseline at context length 4,000 (MASE 0.8666), suggesting that input preconditioning can substitute for doubling the context window.

I. Matched Learning Rate

The baseline is sensitive to learning rate: increasing to $\eta = 2 \times 10^{-3}$ improves it by 2.3% (from 0.8617 to 0.8422). d4_dropout is insensitive (0.1% difference: 0.8368 at 10^{-3} vs. 0.8361 at 2×10^{-3}).

J. Multi-Scale Variants

We test two multi-scale variants that provide two preconditioning channels (degree 4 + degree 6):

All three methods are essentially tied head-to-head (d4_dropout vs MS46: 251/485 wins; d4_dropout vs MSHD10: 257/485 wins; both close to the 50% chance line). MSHD10 has the lowest cross-seed standard deviation (0.0072 vs d4_dropout's 0.0083).

MSHD10 per-seed results:

K. 100K Extended Training

Two findings stand out. First, the benefit of polynomial preconditioning grows with training budget: d4_dropout improves by 3.9% at 100K (vs. 2.9% at 10K). Second, the capacity controls that were near-neutral at 10K now actively hurt: Duplicate degrades by 0.9% and Zero by 1.7%. The baseline shows higher variance across seeds at 100K (seed 0 reaches 0.9092 due to late-training instability), while d4_dropout remains stable (range 0.83–0.86).

L. Inference-Time Ablation

A model trained with Chebyshev preconditioning (d4_dropout) is evaluated with different channel content at inference time:

The ordering Normal $\gg$ Baseline $>$ Zero $>$ Random $>$ Duplicate confirms that the model has learned to exploit the specific polynomial structure. Random and duplicate channels are worse than no preconditioning at all, demonstrating that the model does not simply use the extra channel for generic capacity.

Method	Location	MASE	vs BL
d4_dropout (input-level)	Before patching	0.8234	-4.99%
Latent $s = 1$	After input projection	1.1538	$+33.1\%$
Latent $s = 4$	After input projection	1.2243	$+41.3\%$

Table 11. Domain analysis (d4_dropout vs BL, 5 seeds pooled, 485 observations).

Domain	n	$\Delta\%$	Wins
Demographics	15	$+9.83\%$	13/15
Transport	55	$+6.94\%$	47/55
Energy	80	$+3.97\%$	56/80
M4 Competition	30	$+3.83\%$	23/30
IT/Cloud	120	$+2.23\%$	81/120
ETT (Industrial)	80	$+1.50\%$	54/80
Retail/Sales	20	$+0.85\%$	14/20
Weather	75	$+0.34\%$	36/75
Healthcare	10	$+0.24\%$	5/10

Zeroed-channel dependence vs dropout rate. Models trained with different channel dropout rates, evaluated with preconditioning replaced by zeros:

Without dropout, the model is catastrophically dependent on the preconditioning channel (36% degradation). With 10% dropout, degradation is moderate (9%). With 30%, the model barely uses preconditioning (2.5% degradation but weaker overall).

M. Comparison to Official Moirai 2.0

Our retrained baseline differs from the official Moirai 2.0-R-Small release due to substantially smaller training compute:

The gap between our retrained models and the official release is expected and does not diminish our findings. The official Moirai 2.0 was trained with Salesforce's full compute budget (details not fully disclosed), while our retraining uses only 10K–100K steps. Our experiments are designed as controlled comparisons: all conditions (Baseline, d4_dropout, Zero, Duplicate) use identical training settings, data, and compute. The relative improvement from input preconditioning is the relevant finding, not the absolute MASE level.

N. FEV-Bench Evaluation

To verify that our findings generalize beyond GIFT-Eval, we evaluate on FEV-Bench (100 tasks), an independent time series forecasting benchmark, and reproduce the polynomial-degree sweep over $d \in \{2, \ldots, 8\}$.

d4_dropout improves geometric-mean MASE by 2.91% on FEV-Bench (5-seed mean 1.2532 ± 0.0064 vs. BL 1.2908) and SQL by 3.09% (1.0236 ± 0.0053 vs. 1.0562), consistent with the GIFT-Eval improvement and confirming the benefit is not benchmark-specific.

The FEV-Bench degree-sweep reproduces the qualitative GIFT-Eval pattern: every $d \geq 2$ beats baseline, the curve is shallow, and the extreme d=2 (single non-trivial lag) trails the rest. The benefit transfers cleanly to an independent benchmark.

O. Matched Official Moirai 2.0 Protocol

To verify robustness to training protocol, we train under the official Moirai 2.0 schedule: 10K warmup steps (vs. our default 1K) and 100K total steps.

d4_dropout improves by 2.3% over the baseline under the official protocol. The improvement is seed-dependent (s0 is marginal, s2 and s7 show clear gains), but the mean effect is consistent with our standard protocol results ($+2.9\%$ at 10K steps, $+3.9\%$ at 100K steps with 1K warmup). This confirms that the benefit of polynomial input preconditioning is not an artifact of our training schedule.

Table 12. Frequency analysis (d4_dropout vs BL, 5 seeds pooled).

Frequency	n	$\Delta\%$	**Wins**
Sub-hourly	180	$+3.40\%$	121/180
Hourly	155	$+3.80\%$	113/155
Daily	75	$+0.76\%$	46/75
Weekly	40	$+0.73\%$	29/40
Monthly	25	$+3.41\%$	13/25
Quarterly	5	$+0.24\%$	3/5
Yearly	5	$+5.92\%$	4/5

Table 13. d4 vs d4_dropout vs Baseline by GIFT-Eval term. Bold marks the better of d4/d4_dropout in each bin.

Bin	n	**BL**	**d4**	**d4_dropout**	**d4** $\Delta\%$	**d4_dropout** $\Delta\%$	**d4 better than d4_dropout**
short	55	0.8403	**0.8252**	0.8282	-1.79%	-1.44%	$24/55$
medium	21	0.8592	0.8301	**0.8231**	-3.39%	-4.20%	$9/21$
long	21	0.9237	0.8802	**0.8744**	-4.71%	-5.34%	$7/21$
ALL	97	0.8618	0.8379	**0.8368**	-2.77%	-2.90%	$40/97$

Table 14. d4 vs d4_dropout vs Baseline by raw prediction-length bin (the GIFT-Eval PL distribution has no configs in $(64, 200]$, so we bin around the natural clusters of the benchmark and the $d \cdot s = 64$ mismatch threshold).

Bin	n	**BL**	**d4**	**d4_dropout**	**d4** $\Delta\%$	**d4_dropout** $\Delta\%$	**d4 better than d4_dropout**
PL ≤ 32	30	1.0237	**1.0074**	1.0098	-1.59%	-1.36%	$10/30$
$32 <$ PL ≤ 64	25	0.6631	**0.6496**	0.6529	-2.04%	-1.55%	$14/25$
$480 \leq$ PL ≤ 600	21	0.8592	0.8301	**0.8231**	-3.39%	-4.20%	$9/21$
PL ≥ 720	21	0.9237	0.8802	**0.8744**	-4.71%	-5.34%	$7/21$

Table 15. Context length analysis (seed 0). The benefit is approximately constant ($\sim$5%) across context lengths.

Context	**BL**	**d4_dropout**	$\Delta\%$	**Wins/97**
512	0.9432	0.8986	-4.73%	62
1,000	0.9166	0.8706	-5.02%	65
2,000	0.8813	0.8331	-5.47%	71
4,000	0.8666	0.8234	-4.99%	66

Table 16. Matched learning rate comparison ($\eta = 2 \times 10^{-3}$, 5 seeds, 485 comparisons).

Seed	**BL@2e-3**	**d4_dropout@2e-3**	$\Delta\%$	**Wins/97**
0	0.8424	0.8339	$+1.01\%$	57
1	0.8442	0.8338	$+1.23\%$	55
2	0.8378	0.8468	-1.07%	46
7	0.8339	0.8350	-0.13%	54
42	0.8526	0.8307	$+2.55\%$	68
Mean	**0.8422**	**0.8361**	$+0.72\%$	**280/485**
Std	0.0071	0.0062	1.38%	—

Table 17. LR $\times$ method interaction (5-seed mean MASE). d4_dropout provides two benefits: optimization stability (LR-insensitive) and genuine inductive bias ($+0.72\%$ at matched optimal LR).

	$\eta = 10^{-3}$	$\eta = 2 \times 10^{-3}$	LR sensitivity
Baseline	0.8617	0.8422	2.27%
d4_dropout (Ours)	0.8368	0.8361	0.09%
d4_dropout vs BL	-2.89% ($329/485$ wins)	-0.72% ($280/485$ wins)	

Table 18. Multi-scale variants (5 seeds, $\eta = 10^{-3}$, 10K steps).

Method	Mean	Std	vs BL	Wins/485
`d4_dropout` (d=4, 10% drop)	0.8368	0.0083	-2.89%	329
MS46 (d=4+6, 0% drop)	0.8434	0.0123	-2.13%	330
MSHD10 (d=4+6, 10% drop)	0.8437	0.0072	-2.09%	315

Seed	MASE	$\Delta\%$	Wins/97
0	0.8555	-1.28%	54
1	0.8342	-1.89%	62
2	0.8447	-2.12%	64
7	0.8453	-2.57%	73
42	0.8388	-2.61%	62
Mean	0.8437	-2.09%	315/485
Std	0.0072	0.49%	—

Table 19. Full 100K training budget with capacity controls (5 seeds). `d4_dropout` vs BL: $+3.9\%$; `d4_dropout` vs Dup: $+4.7\%$; `d4_dropout` vs Zero: $+5.5\%$.

	s0	s1	s2	s7	s42	Mean
Baseline	0.9092	0.8920	0.8633	0.8641	0.8636	0.8784
`d4_dropout`	**0.8349**	**0.8402**	0.8586	**0.8450**	**0.8430**	**0.8443**
MSHD10	0.8266	—	**0.8432**	0.8437	—	0.8378[†]
Duplicate	0.8887	0.8740	0.8974	0.8794	0.8903	0.8860
Zero	0.8917	0.8585	0.8750	0.8825	0.9610	0.8937
`d4_dropout` $\Delta\%$ vs BL	-8.2%	-5.8%	-0.5%	-2.2%	-2.4%	-3.9%

[†]MSHD10 mean over 3 available seeds (0, 2, 7).

Table 20. Inference-time content ablation (seed 0, `d4_dropout` model). The ordering confirms the model has learned to exploit specific polynomial structure.

Inference channel content	MASE	vs Normal
Normal (Chebyshev)	0.8234	—
No preconditioning channel (Baseline arch)	0.8666	$+5.2\%$
Zeros	0.9003	$+9.3\%$
Random Gaussian noise	0.9427	$+14.5\%$
Duplicate (copy of input)	1.0699	$+29.9\%$

Train dropout	Normal MASE	Zeroed-channel MASE	Degradation
0% (MS46)	0.8534	1.1594	$+35.9\%$
2%	0.8388	0.8996	$+7.2\%$
10% (`d4_dropout`)	0.8234	0.9003	$+9.3\%$
30%	0.8559	0.8769	$+2.5\%$

Table 21. Comparison to official Moirai 2.0-R-Small (HuggingFace release). The gap is expected: the official model was trained with Salesforce's full compute budget.

Model	Geo Mean MASE	Configs < 1.0	Training
Official Moirai 2.0-R-Small (HF)	0.7280	57/97	Full Salesforce budget
Our BL (5-seed mean, 10K)	0.8617	—	10K steps
Our `d4_dropout` (5-seed mean, 10K)	0.8368	—	10K steps
Our BL (5-seed mean, 100K)	0.8784	—	100K steps
Our `d4_dropout` (5-seed mean, 100K)	0.8443	—	100K steps

Table 22. FEV-Bench headline. `d4_dropout` (d=4) is the 5-seed mean; baseline is single-seed. SQL is FEV-Bench's scaled quantile loss.

Method	Geo Mean MASE	SQL	Wins/100
`d4_dropout` (Ours, 5 seeds)	**1.2532**	**1.0236**	**72/100**[†]
Baseline (1 seed)	1.2908	1.0562	—

[†]Win count from single-seed `d4_dropout` vs single-seed BL; the 5-seed mean MASE matches the single-seed value to within rounding.

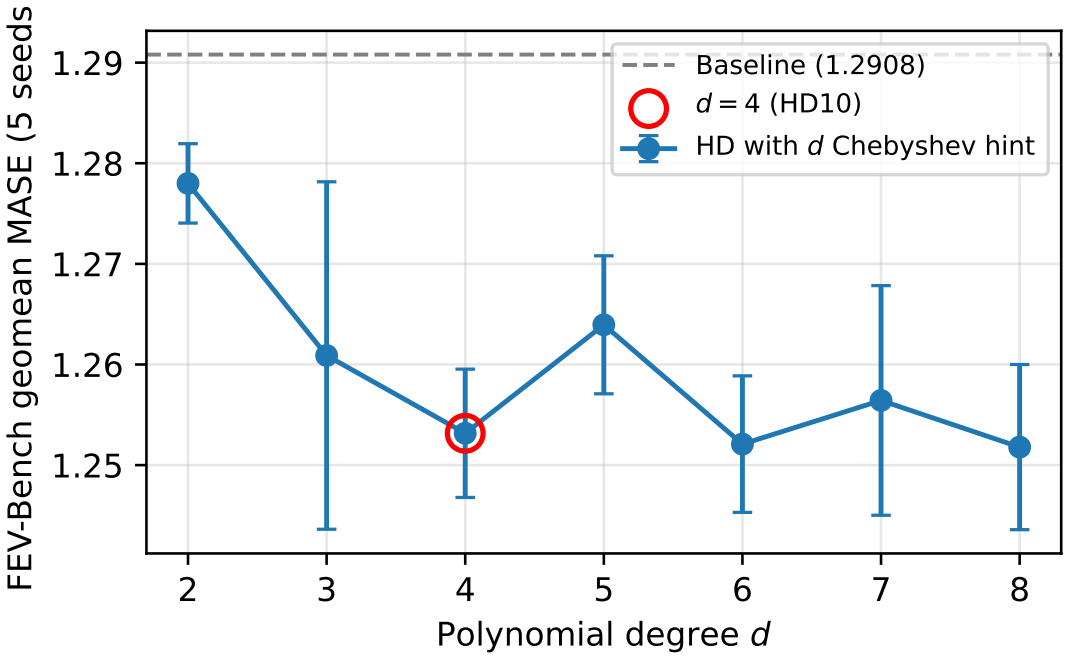

Figure 5. FEV-Bench polynomial-degree sweep (5 seeds per d, stride s=16, 10% channel dropout). Errorbars are cross-seed std; dashed line is the single-seed baseline. Every $d \geq 2$ beats baseline; the curve is shallow with $d \in \{4, 6, 8\}$ statistically tied at the optimum, mirroring the GIFT-Eval pattern (Fig. 2).

Table 23. FEV-Bench polynomial-degree sweep, 5 seeds per degree. $\Delta\%$ relative to the single-seed baseline (1.2908).

Degree	MASE (mean $\pm$ std)	$\Delta\%$ vs BL	SQL (mean $\pm$ std)
2	1.2780 ± 0.0039	-0.99%	1.0425 ± 0.0032
3	1.2609 ± 0.0173	-2.32%	1.0288 ± 0.0125
4	$\mathbf{1.2532 \pm 0.0064}$	$\mathbf{-2.92\%}$	$\mathbf{1.0236 \pm 0.0053}$
5	1.2639 ± 0.0069	-2.08%	1.0309 ± 0.0063
6	1.2521 ± 0.0068	-3.00%	1.0236 ± 0.0060
7	1.2564 ± 0.0114	-2.66%	1.0263 ± 0.0094
8	1.2518 ± 0.0082	-3.02%	1.0228 ± 0.0059

Table 24. Matched official Moirai 2.0 protocol (10K warmup, 100K steps, 3 seeds). Normalized MASE (lower is better).

Method	s0	s2	s7	Mean
Baseline (10K warm)	0.882	0.880	0.917	0.893
`d4_dropout` (10K warm)	**0.892**	**0.846**	**0.880**	**0.873**
$\Delta\%$	-1.1%	$+3.9\%$	$+4.0\%$	$+2.3\%$